PROJECTMARKET · FOUNDATION

Repository Overview

How to navigate the ProjectMarket repository

INTERNAL DOCUMENT

2026-05-15

Two-sided marketplace platform. Romania-first launch, international-ready architecture.

This is a **foundation-phase** repository. The product concept — services marketplace (stylists, tattoo artists, beauty, fashion consultants) vs. handmade goods marketplace — is undecided. The codebase uses concept-neutral terminology so that either can be specialised later without renaming.

Status

Phase 0 — Foundation. Planning, architecture, and design documentation are in place. A Next.js application skeleton is wired with i18n, light/dark theme, shadcn/ui primitives, a concept-agnostic landing page with a waitlist UI, and stub routes for the major authenticated surfaces. No backend services are connected yet. See docs/roadmap.md.

Quick start

```
npm install
npm run dev
# open http://localhost:3000  (redirects to /ro)
```

Default locale is Romanian (`/ro`); English is available at `/en` . The toggle in the header switches between them.

Scripts

Command	Effect
`npm run dev`	Start the Next.js dev server on port 3000
`npm run build`	Production build (typechecks + lints + static-generates routes)
`npm run start`	Run the production build
`npm run lint`	Lint with Next.js's ESLint config
`npm run docs:pdf`	Regenerate `docs/pdf/*.pdf` from the markdown sources

Repository layout

```
projectMarket/
├── README.md
├── docs/                    # Planning and architecture documentation
│   ├── pdf/                 # Generated PDFs (regenerate with npm run docs:pdf)
│   └── diagrams/            # Mermaid and Excalidraw diagrams
├── tools/
│   └── docs-build/          # Node script that builds the documentation PDFs
├── src/
│   ├── app/                 # Next.js App Router
│   │   ├── globals.css
│   │   └── [locale]/        # Locale-prefixed routes (ro, en)
│   │       ├── layout.tsx
│   │       ├── page.tsx     # Landing page
│   │       ├── not-found.tsx
│   │       ├── (auth)/      # /login, /signup
│   │       └── (app)/       # /browse, /listing/[id], /profile/[handle], /dashboard
│   ├── components/
│   │   ├── ui/              # shadcn/ui primitives (button, input, card)
│   │   └── ...              # site-header, footer, locale-switcher, theme-toggle, e
│   ├── i18n/                # next-intl config (routing, request, navigation)
│   ├── lib/                 # Shared utilities (cn helper)
│   ├── messages/            # Locale message bundles (ro.json, en.json)
│   └── middleware.ts        # next-intl locale routing middleware
├── next.config.mjs
├── tailwind.config.ts
├── tsconfig.json
├── components.json          # shadcn/ui configuration
├── .env.example             # All env vars catalogued (services not yet wired)
└── package.json
```

Where to start reading

1. Product overview — what this is and who it's for
2. Tech stack — what we're building with, and why
3. Architecture — system shape and the concept-specific seam
4. Services and APIs — third-party services and pricing
5. Roadmap — phased plan and idea catalogue
6. Glossary — concept-neutral terminology
7. Design brief — direction for Figma work

Market research:

- Beauty services concept — preliminary research from the marketing partner; translated from Romanian

PDFs of every document live in docs/pdf/ for sharing outside the repo.

What is and isn't wired

Wired:

- Next.js 15 App Router + TypeScript + Tailwind CSS + shadcn/ui
- next-intl with Romanian (default) and English locales, locale-prefixed routes, locale switcher in the header
- next-themes with system / light / dark mode toggle
- Landing page with a UI-only waitlist form (shows a thank-you state on submit but does not persist anything)
- Stub routes for `/login`, `/signup`, `/browse`, `/listing/[id]`, `/profile/[handle]`, `/dashboard`, `/messages`
- 404 page

Not wired (intentionally — see Roadmap):

- Authentication (no Supabase project connected; auth forms are visual stubs)
- Database (no Supabase keys configured)
- Waitlist persistence (form is UI-only; real capture lands when email service is wired)
- Email / Stripe / Sentry / analytics — all listed in `.env.example` but commented out
- Concept-specific code (`src/concepts/services/`, `src/concepts/goods/` — created when the concept is chosen)

Working conventions

- All code, comments, identifiers, and documentation are written in **English**.
- End-user-facing UI strings are translated; **Romanian** is the launch locale, with i18n scaffolding from day one.
- Currency amounts are stored as integers in **minor units** with an explicit currency code.
- All timestamps are stored in **UTC**; display is localised.
- Concept-specific terminology (`Expert`, `Service`, `Artisan`, `Product`) does not appear in identifiers. See Glossary.

Regenerating the documentation PDFs

PDFs are rebuilt from the markdown sources via a self-contained Node script in `tools/docs-build/`. Mermaid diagrams are rendered to SVG by a headless Chrome instance during the build.

```
npm run docs:pdf
```

Requires Google Chrome to be installed at the default macOS path (`/Applications/Google Chrome.app`). If you run this elsewhere, edit `tools/docs-build/build-pdf.js` and point `CHROME_PATH` at your Chrome / Chromium binary.

Foundation phase: what's next

The next step is **concept-lock** — your partner picks services vs. goods. At that point:

1. The Listing / Transaction / Fulfillment seam in docs/architecture.md gets populated in `src/concepts/<chosen>/`.
2. A real Supabase project is provisioned and `NEXT_PUBLIC_SUPABASE_*` vars filled in.
3. The auth flow is wired and the stub routes start showing real data.

Until then, everything in this repo can be iterated on without committing to a concept.